

Recuit simulé & Recherche Tabou

Aperçu

- Introduction
- Algorithme du recuit simulé
- Problème de l'affectation quadratique
- Recherche Tabou

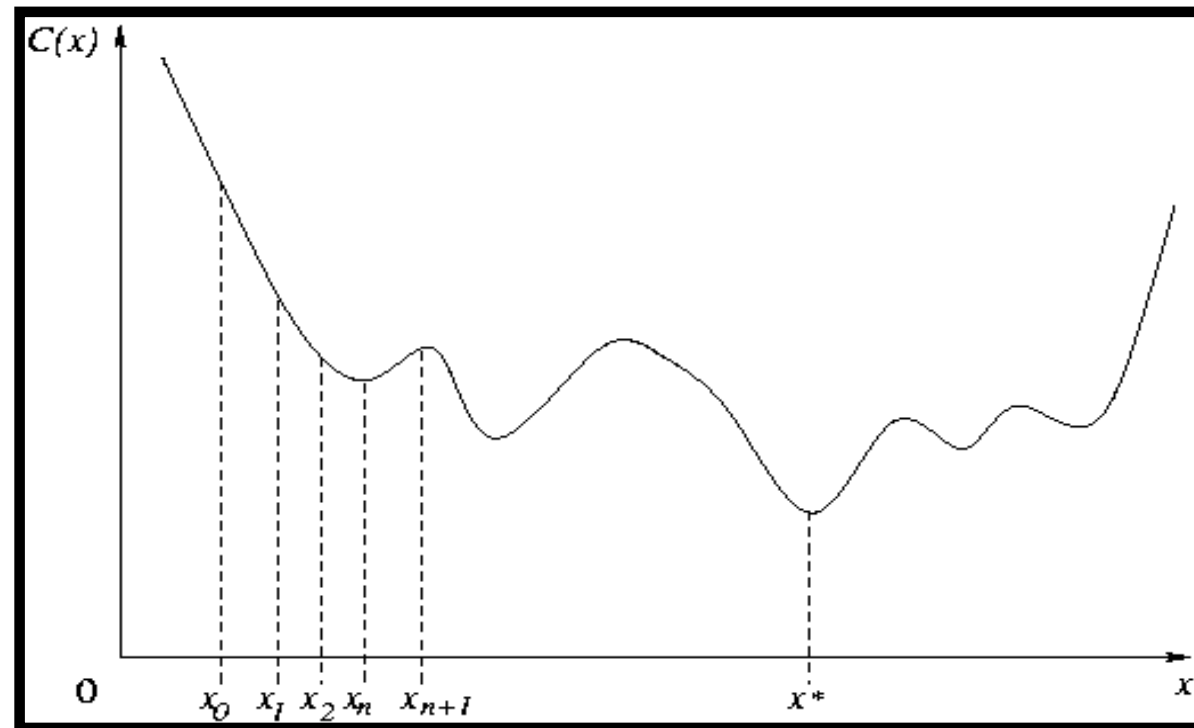
Problème d'optimisation

- Ω : **espace des configurations**
ou **espace de recherche**
- C : **fonction de coût** (ou objectif ou fitness)
- Trouver une configuration $x^* \in \Omega$ de coût minimal

$$C(x^*) = \min \{ C(x) \mid x \in \Omega \}$$

Piège des minima locaux

- But: trouver une stratégie pour pouvoir sortir **des minima locaux**

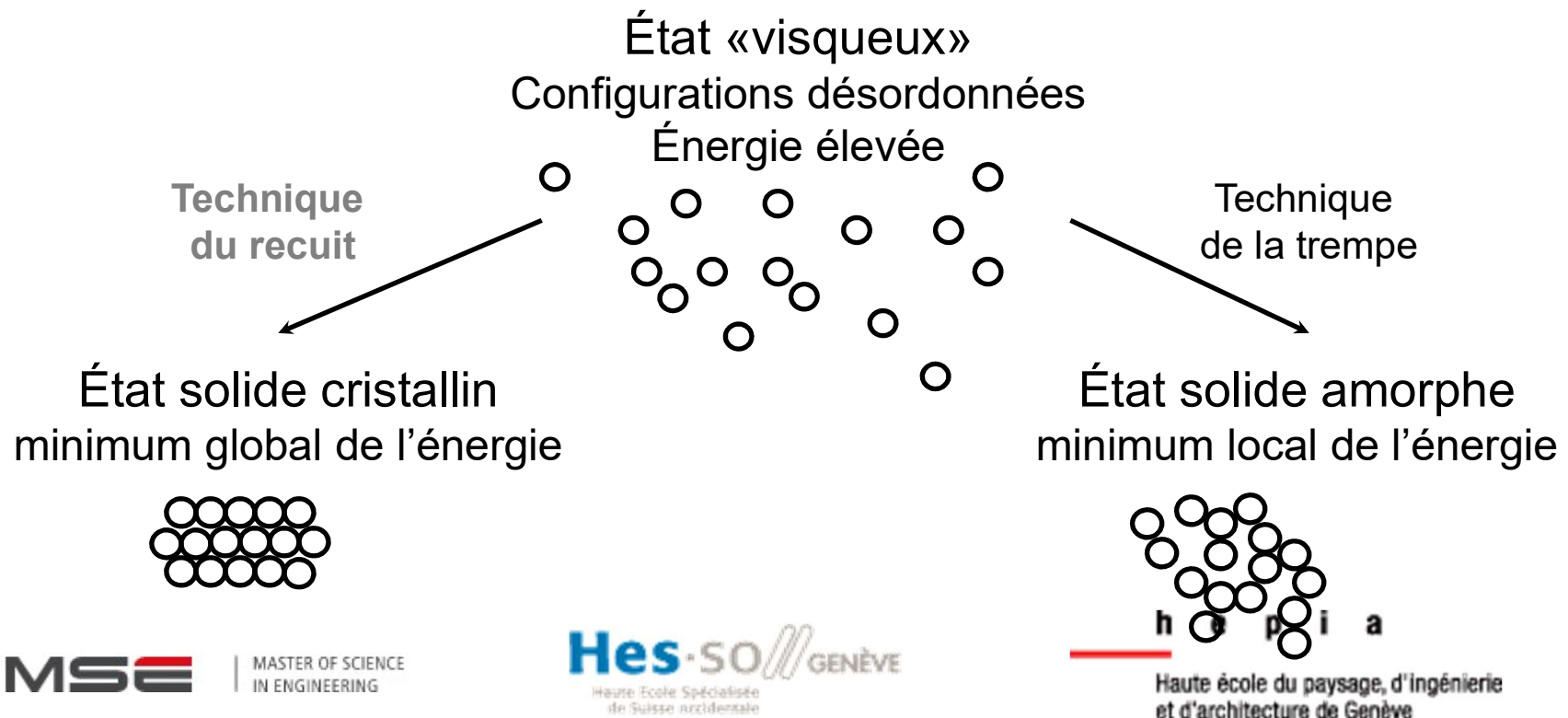


Le recuit simulé (« Simulated Annealing »)

Origine du recuit simulé

Origine du recuit simulé (*simulated annealing*)

Recuit métallurgique ou cristallisation d'un liquide



Principe du recuit

- Initialement le métal est porté à haute température
- Puis refroidissement progressif
 - à haute température
 - atomes très agités (configurations atomiques équiprobables)
 - à basse température
 - atomes organisés en une structure atomique parfaite (configuration proche de l'état d'énergie minimale)
- Contrainte
 - Refroidissement lent
 - Si le refroidissement est trop rapide, il y a un risque de rester bloqué dans un minimum local (configuration sous-optimale)

Analogie pour le recuit simulé

- Analogie problème d'optimisation / système physique

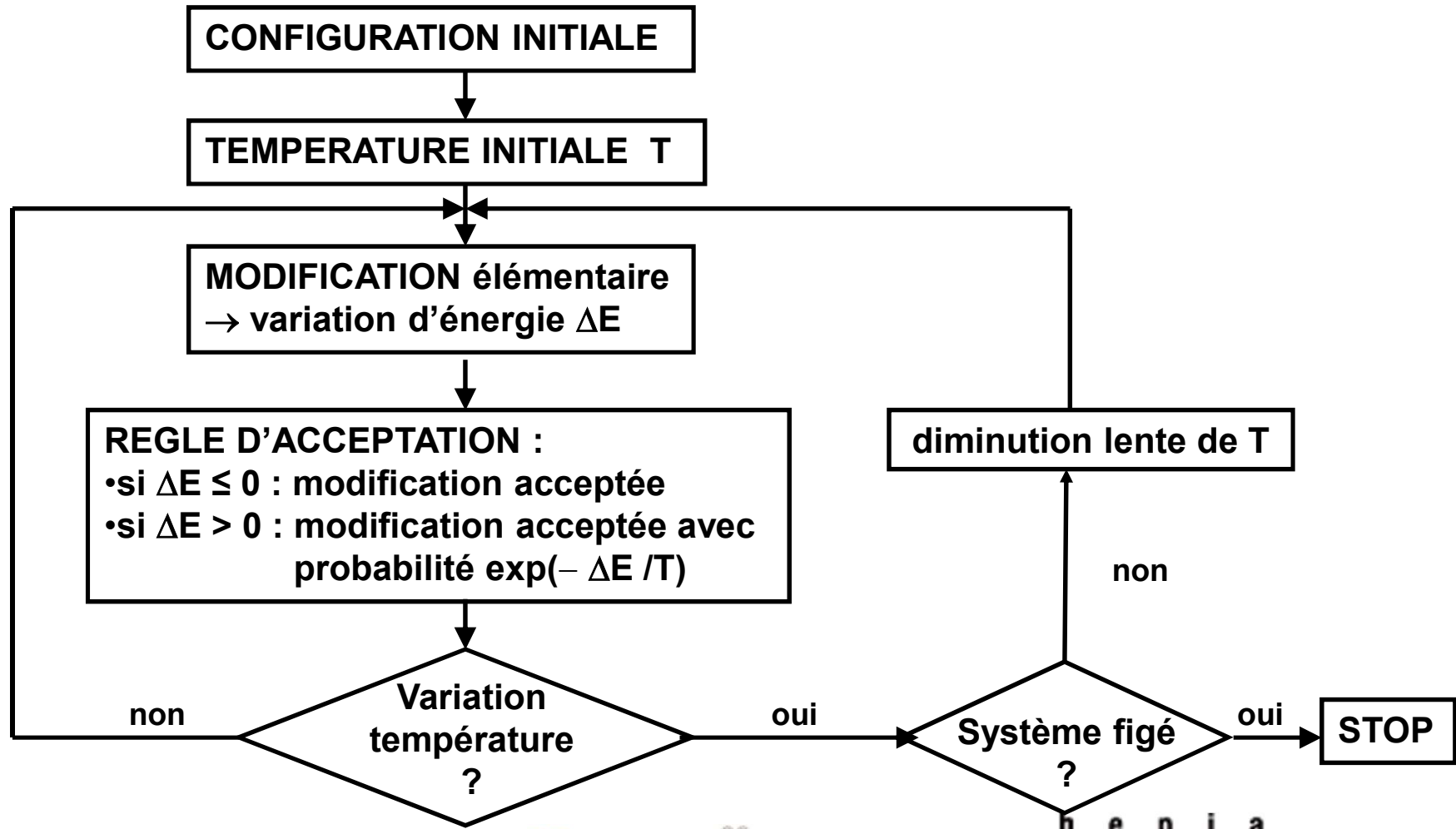
Problème d'optimisation	Système physique
fonction de coût / objectif $C(x)$	énergie libre $E(X)$
variables du problème	"coordonnées" des atomes
trouver une "bonne" configuration	trouver un état de basse énergie

- Algorithme du recuit simulé (Kirkpatrick & al. - 1983)

Paramètre de température

- Algorithme
 - Le paramètre **température** (agitation thermique) autorise avec une certaine probabilité le choix de configurations d'énergie plus élevée
 - ⇒ capacité d'éviter les minima locaux
 - Suite de configurations dont le choix et l'acceptation dépendent de la fonction objectif et de la température
 - Procédure de refroidissement qui donne la décroissance de la température au cours du temps

Recuit simulé : organigramme



Algorithme générique

- Algorithme du recuit simulé (version Metropolis)

$T \leftarrow T_0$ -- Température initiale

$X \leftarrow X_0$ -- Configuration initiale

répéter

Tirer aléatoirement $Y \in \text{Voisinage}(X)$

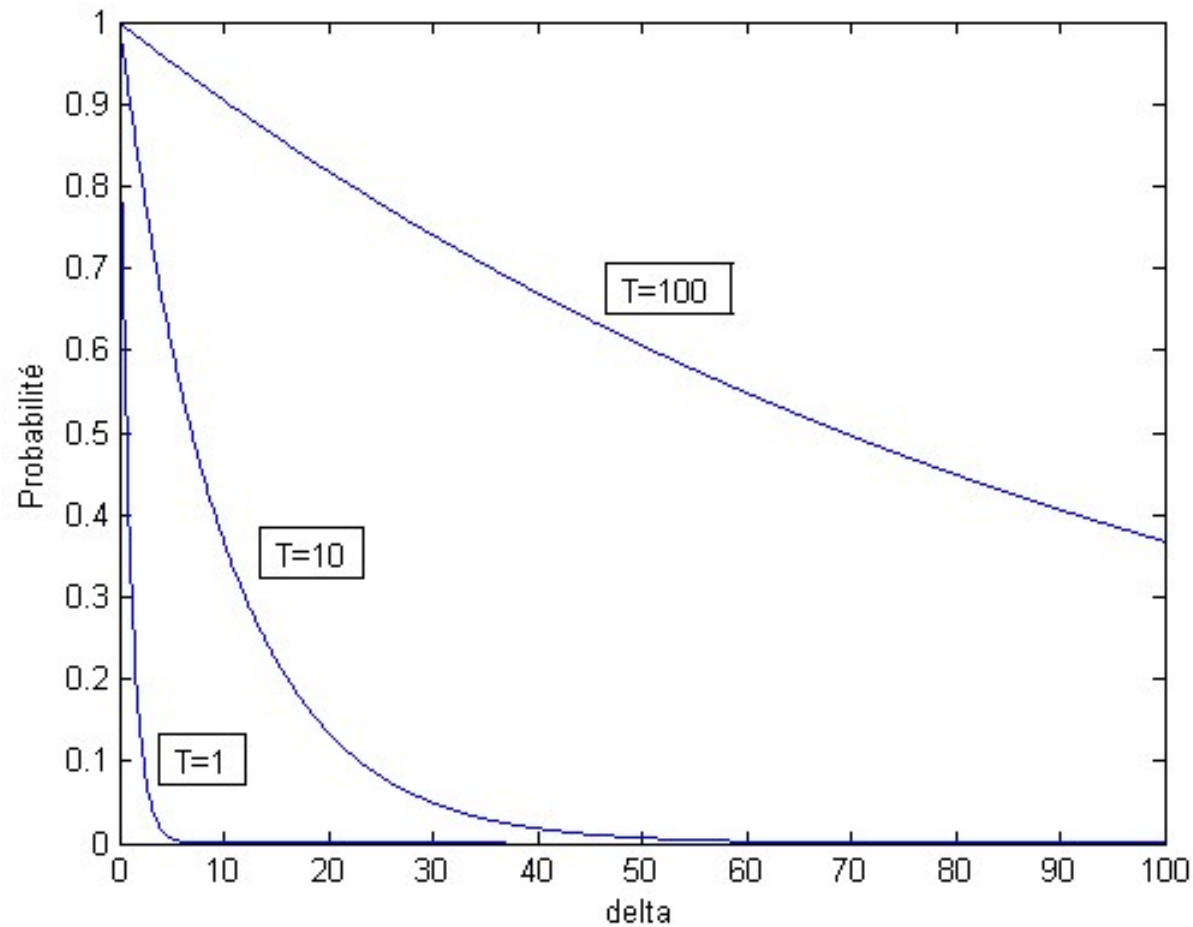
si $\Delta E = E(Y) - E(X) < 0$ ou $\exp(-\Delta E / T) > \mu$, $\mu \in [0;1]$ aléatoire

alors $X \leftarrow Y$

$T \leftarrow g(T)$ -- refroidissement : g strictement décroissante

jusqu'à critère d'arrêt vérifié

Probabilité d'acceptation des sauts positifs



Variation de la température

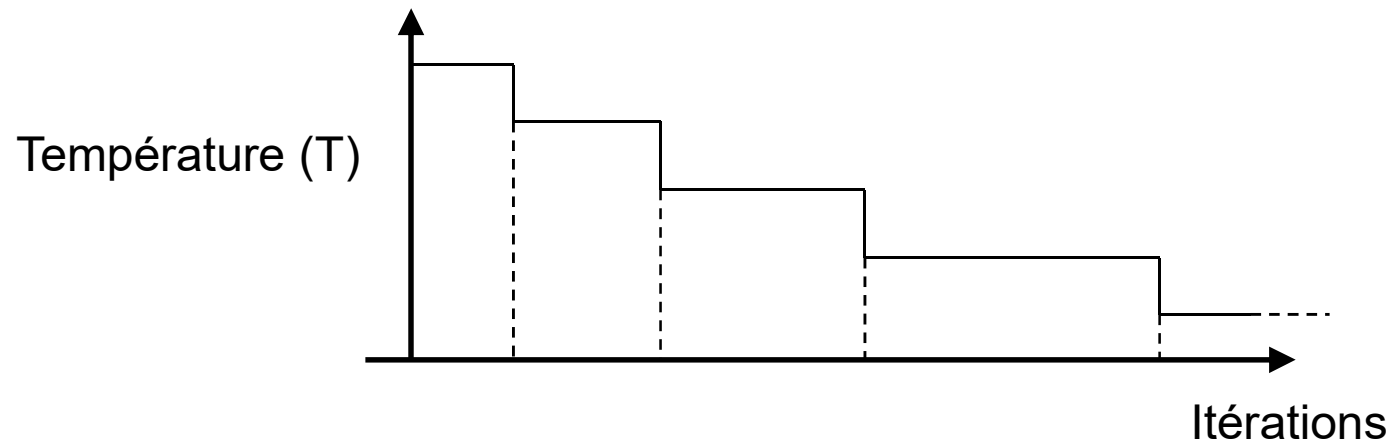
- Choix des paramètres (en pratique)
 - T_0 est choisie à l'issue d'expérimentations ou tests
 - Schéma de température exponentiel

$$T_n = T_0 \cdot \alpha^n, \text{ avec } n \in \{0,1,\dots\} \text{ et } 0 < \alpha < 1$$

- Critères d'arrêt possibles
 - pourcentage de configurations acceptées en dessous d'un seuil fixé
 - variation de l'énergie trop faible
 - choix d'une température minimale

Variation de la température par paliers

- Metropolis et al. (1953)
- Phénomènes de recuit thermodynamique
- Schéma de refroidissement par paliers



Propriété de convergence

Remarques

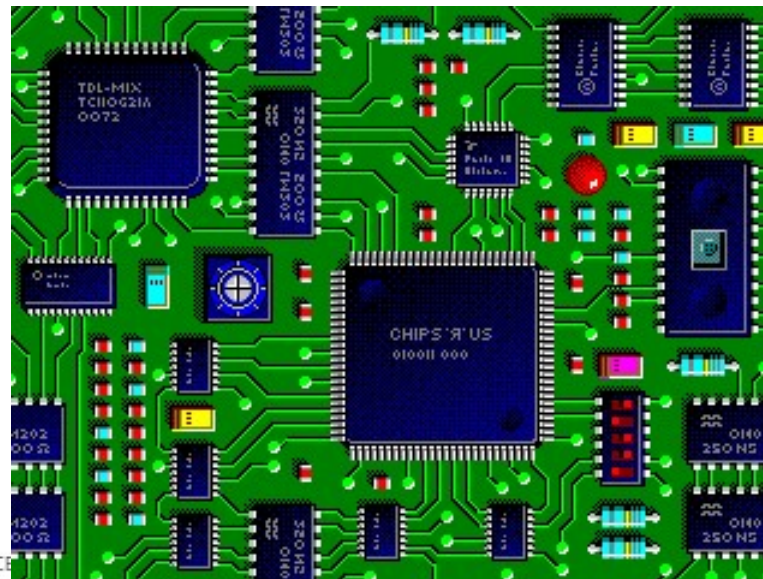
- seule métaheuristique pour laquelle il existe une preuve de convergence à l'optimum
 - sous certaines hypothèses
 - si $T \rightarrow 0$ en même temps que le nombre d'itérations $\rightarrow \infty$
- a suscité beaucoup d'intérêt fin 80' et début 90' à cause de la "convergence garantie"
- robuste et relativement efficace si structure de voisinage bien choisie
- facile à mettre en œuvre

Avantages / Inconvénients

- Avantages
 - Solutions de bonnes qualités
 - Méthode générale applicable à tout problème d'optimisation
 - Fonction à optimiser évaluable
 - Notion de voisins garantissant la connexité
 - Facile à programmer
 - Nouvelles contraintes incorporables à tout moment (e.g. problème des horaires)
- Inconvénients
 - Temps de calcul
 - Recherche de la solution exacte $\Rightarrow$ temps de calcul exponentiel
 - Recherche d'une solution approchée à 2% $\Rightarrow$ temps de calcul en $O(N^3)$; N étant la taille du problème

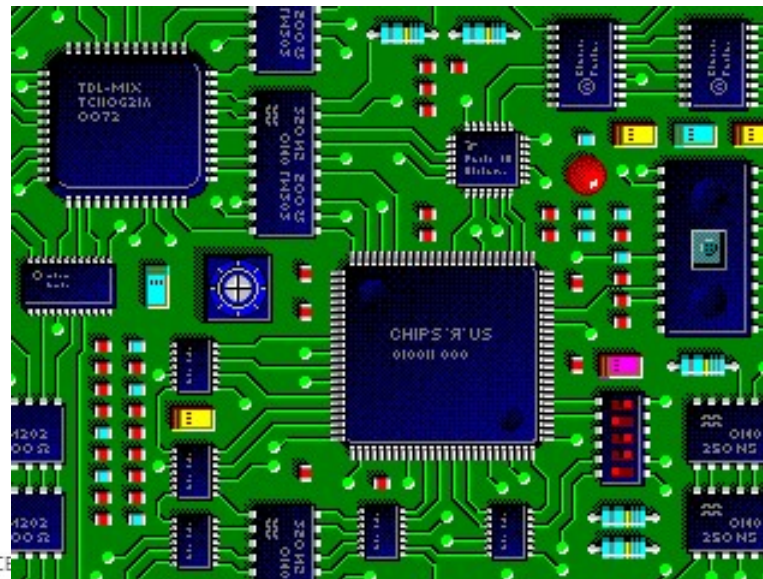
Le recuit simulé: application

- Placement de composants électroniques
 - Fonction à optimiser?
 - Comment formuler le problème?
 - Quel algorithme?



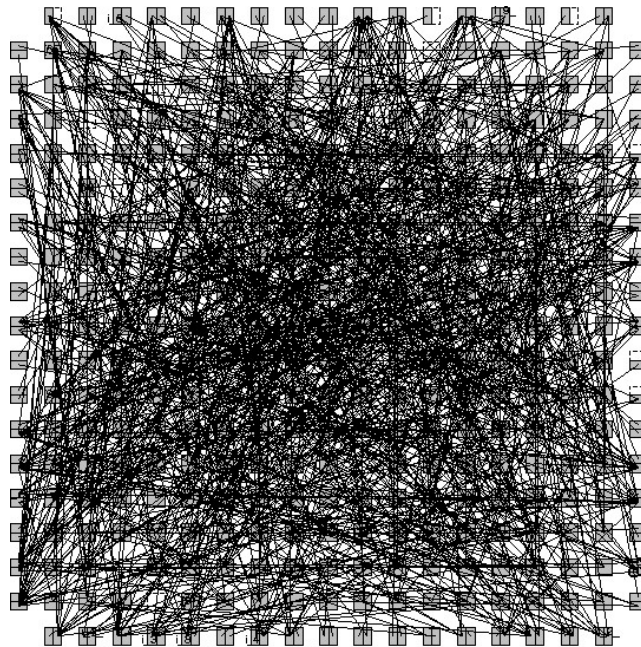
Le recuit simulé: application

- Placement de composants électroniques
 - Modules → sommets d'un graphe
 - Interconnexions → arêtes d'un graphe
 - Coût de l'arrangement
 - longueur totale des fils + aire totale occupée



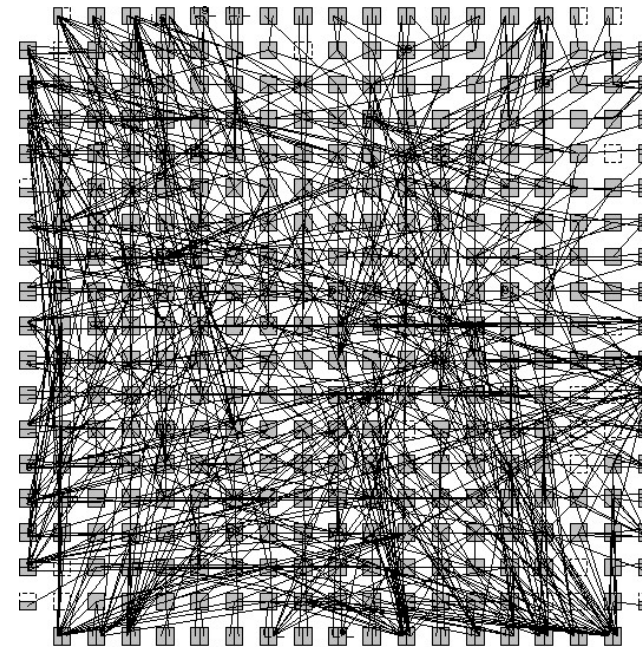
Placement sur une FPGA

Placement initial aléatoire



Initial Placement. Cost: 74.5562. Channel Factor: 100

Placement final

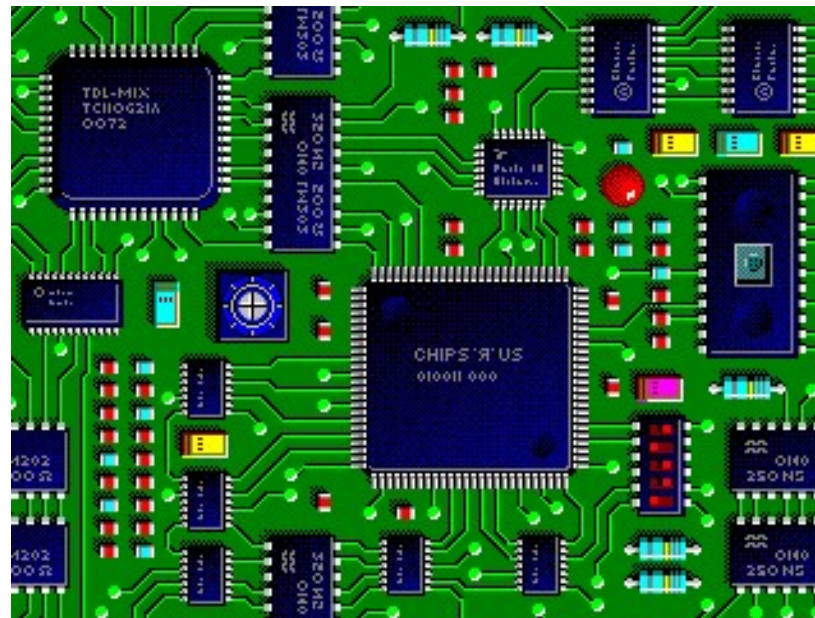


Final Placement. Cost: 28.5384. Channel Factor: 100

- Les emplacements possibles sont fixes ici
- Donc pas d'optimisation de l'aire totale

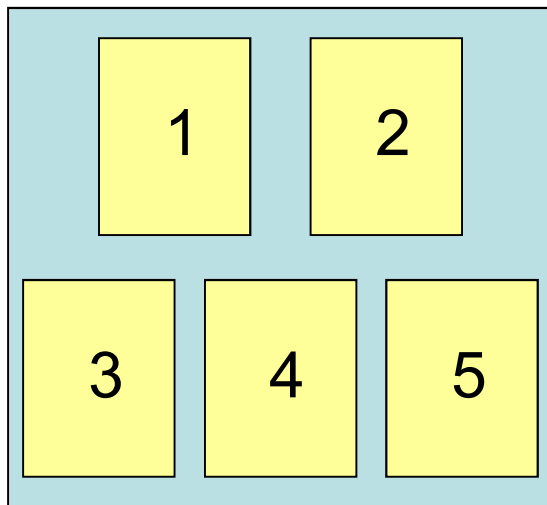
Problème de l'affectation quadratique

Problème de l'affectation quadratique



Affectation quadratique

5 emplacements



Distances entre les emplacements

	1	2	3	4	5
1	0	1	1	2	3
2	1	0	2	1	2
3	1	2	0	1	2
4	2	1	1	0	1
5	3	2	2	1	0

Affectation quadratique

5 modules (objets)

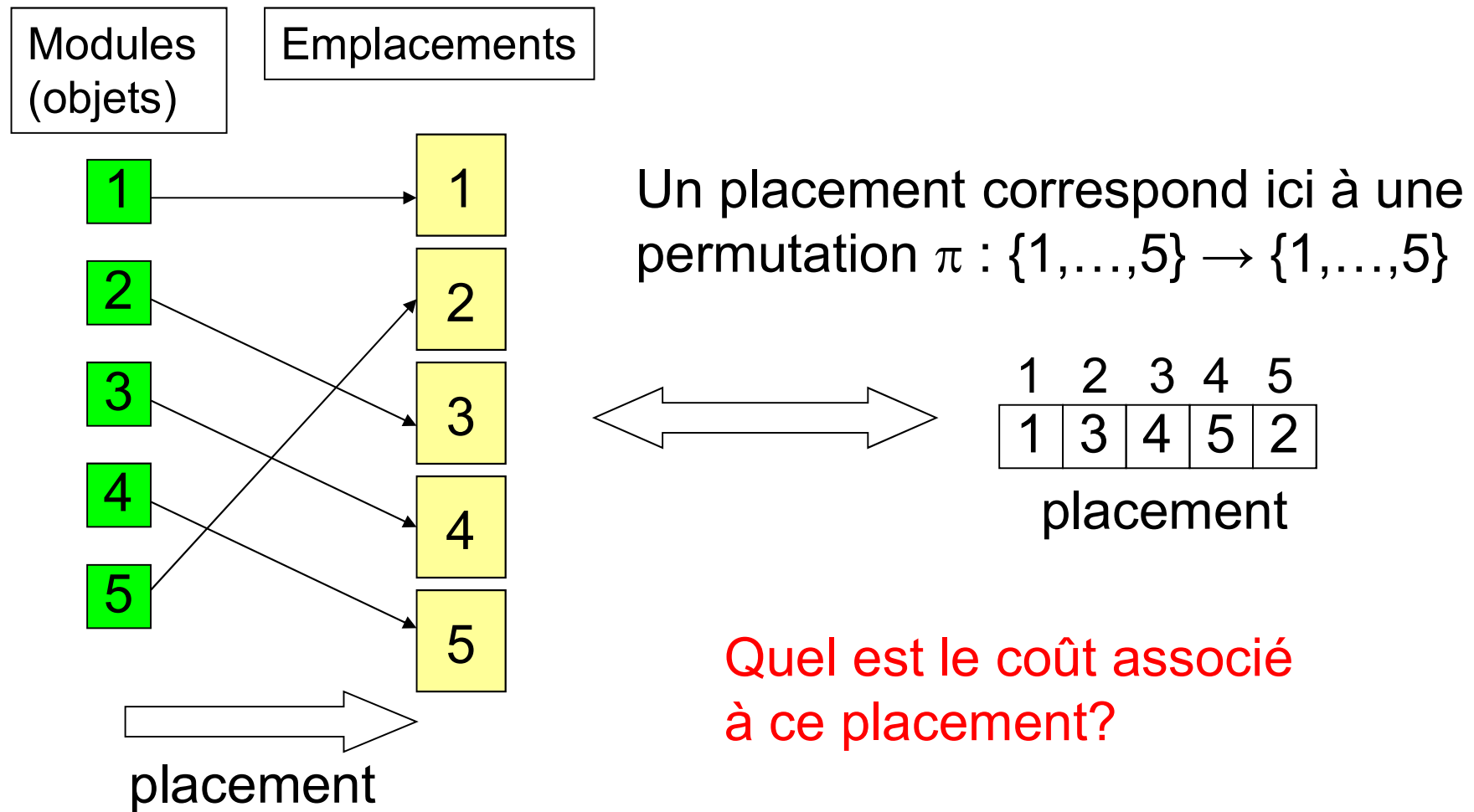
1 2 3 4 5

Poids des connexions
entre les objets

	1	2	3	4	5
1	0	5	2	4	1
2	5	0	3	0	2
3	2	3	0	0	0
4	4	0	0	0	5
5	1	2	0	5	0

Rem : pas forcément symétrique

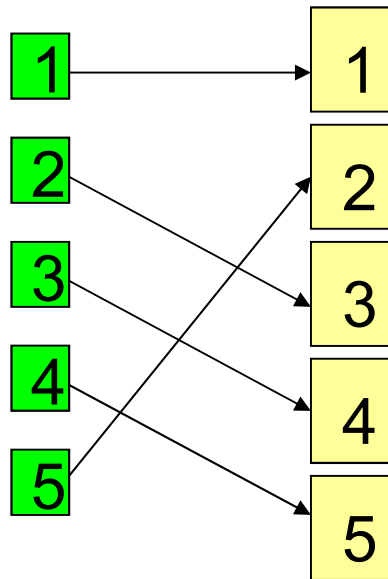
Affectation quadratique



Affectation quadratique

Objets

Emplacements



$\pi :$

1	2	3	4	5
1	3	4	5	2

 placement

Matrice F
des flots

	1	2	3	4	5
1	0	5	2	4	1
2	5	0	3	0	2
3	2	3	0	0	0
4	4	0	0	0	5
5	1	2	0	5	0

Coût associé?

$$\sum_{i,j=1}^5 F(i, j) \cdot D(\pi(i), \pi(j))$$

Matrice D
des distances

	1	2	3	4	5
1	0	1	1	2	3
2	1	0	2	1	2
3	1	2	0	1	2
4	2	1	1	0	1
5	3	2	2	1	0

Affectation quadratique

- Données
 - n objets, flots $F(i, j)$ entre les objets i et j
 - n emplacements, distances $D(r, s)$ entre les emplacements r et s
- Formulation du problème
 - Placer les n objets sur les n emplacements de manière à minimiser la somme des produits flots $\times$ distances
 - Espace des solutions : $n!$ permutations
 - Mathématiquement, on cherche une permutation π , dont la $k^{\text{ème}}$ composante donne la place de l'objet k , et qui minimise

$$\sum_{i,j=1}^n F(i, j) \cdot D(\pi(i), \pi(j))$$

Affectation quadratique

- Quelques applications
 - Répartition de bâtiments ou de services (hôpitaux)
 - Flots : fréquences des déplacements des personnes d'un bâtiment à l'autre
 - Portes d'embarquement dans un aéroport
 - Flots : nombre de personnes devant transiter d'une porte d'embarquement à une autre
 - Placement de modules dans des circuits électroniques
 - Flots : nombre de connexions entre deux modules
 - Répartition des fichiers dans une base de données
 - Flots : probabilité de demander l'accès au second fichier si on accède au premier

Recherche avec Tabou

Recherche avec tabou

- Heuristique de recherche locale pour résoudre des problèmes complexes ou de très grande taille
- Glover (1986) ; Indépendamment, Hansen (1986)
- Principe de base
 - Recherche de solutions au-delà d'un optimum local
 - En permettant des déplacements qui dégradent la solution
 - En utilisant le **principe de mémoire** pour éviter les retours en arrière (mouvements cycliques)

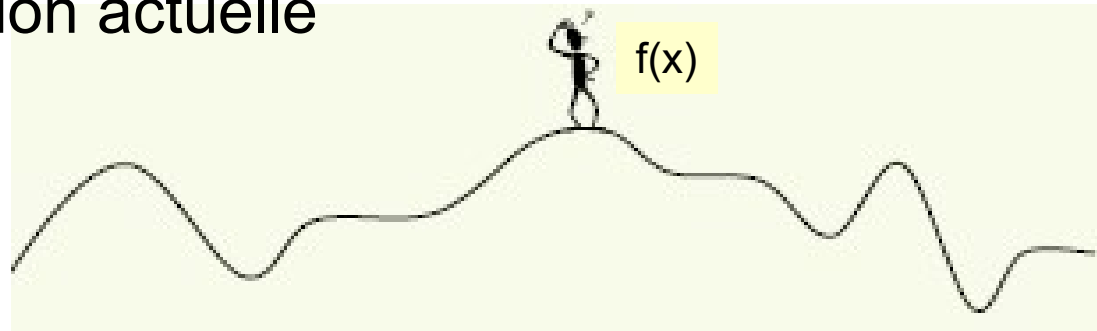
Recherche avec tabou

- **Mémoire**
 - Représentée par une liste taboue qui contient les mouvements ou solutions **temporairement** interdits
 - Rôle au cours de la résolution
 - Diversification (exploration de l'espace des solutions)
- **Exception aux interdictions**
 - Possible de violer une interdiction lorsqu'un mouvement interdit permet d'obtenir la meilleure solution enregistrée

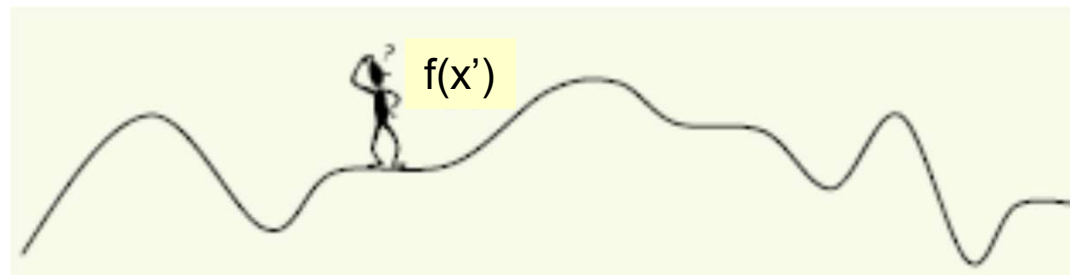
Recherche avec tabou

- Variables du problème

- x : la solution actuelle

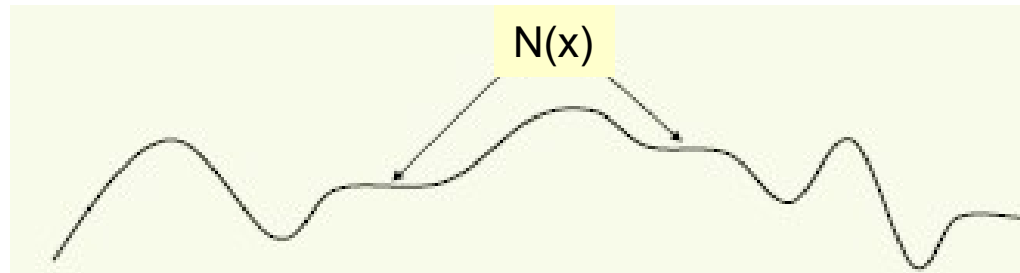


- x' : la prochaine solution atteinte (solution voisine)

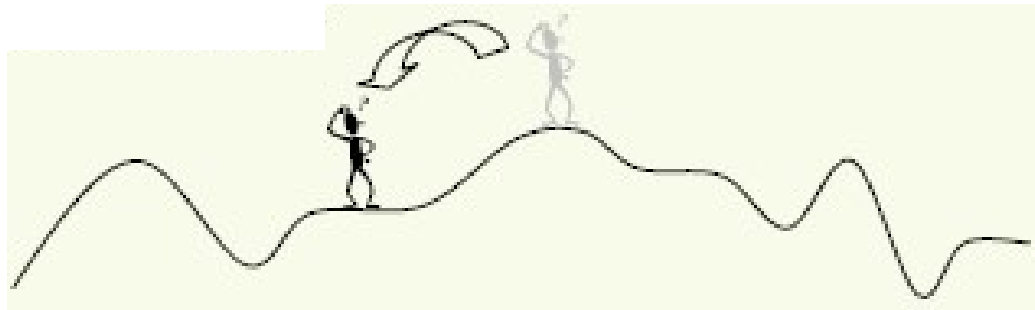


Recherche avec tabou

- Variables du problème
 - $N(x)$: l'espace de solutions voisines à x

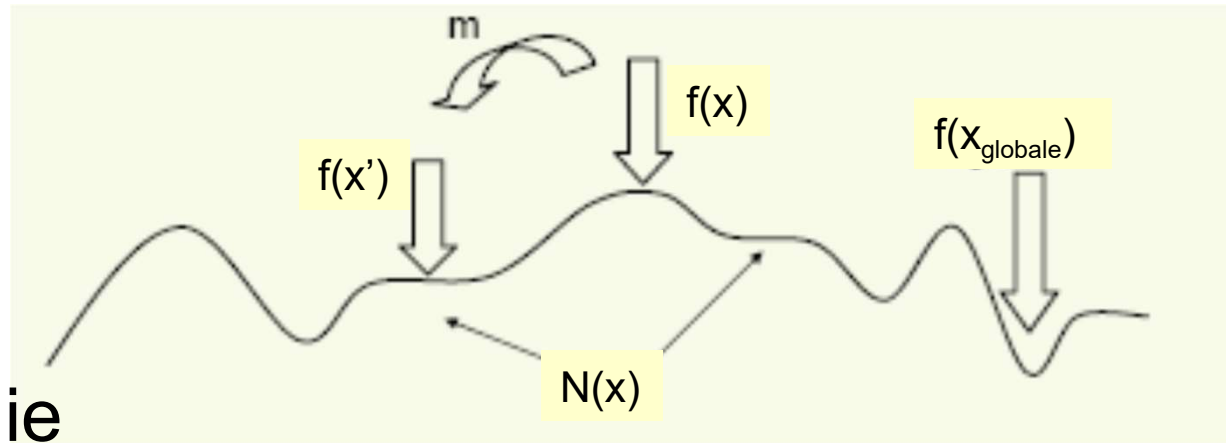


- m : mouvement de x à x'



Recherche avec tabou

- Situation

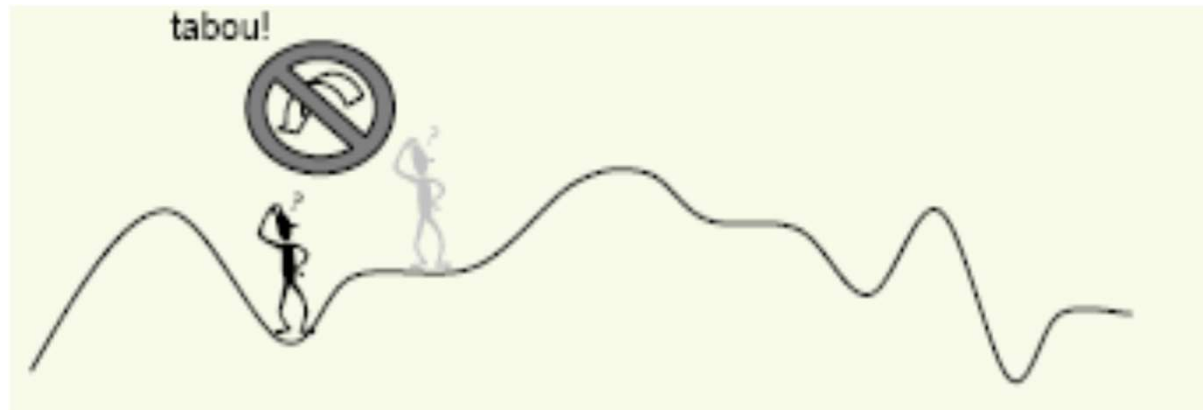


- Terminologie

- Différence : la recherche tabou choisira le meilleur x' dans $N(x)$ l'ensemble des solutions voisines
- Mouvement non améliorateur
 - Sortir d'un minimum local vers une solution voisine *pire*
 - La méthode tabou permet un mouvement non améliorateur, comme le permet le recuit simulé

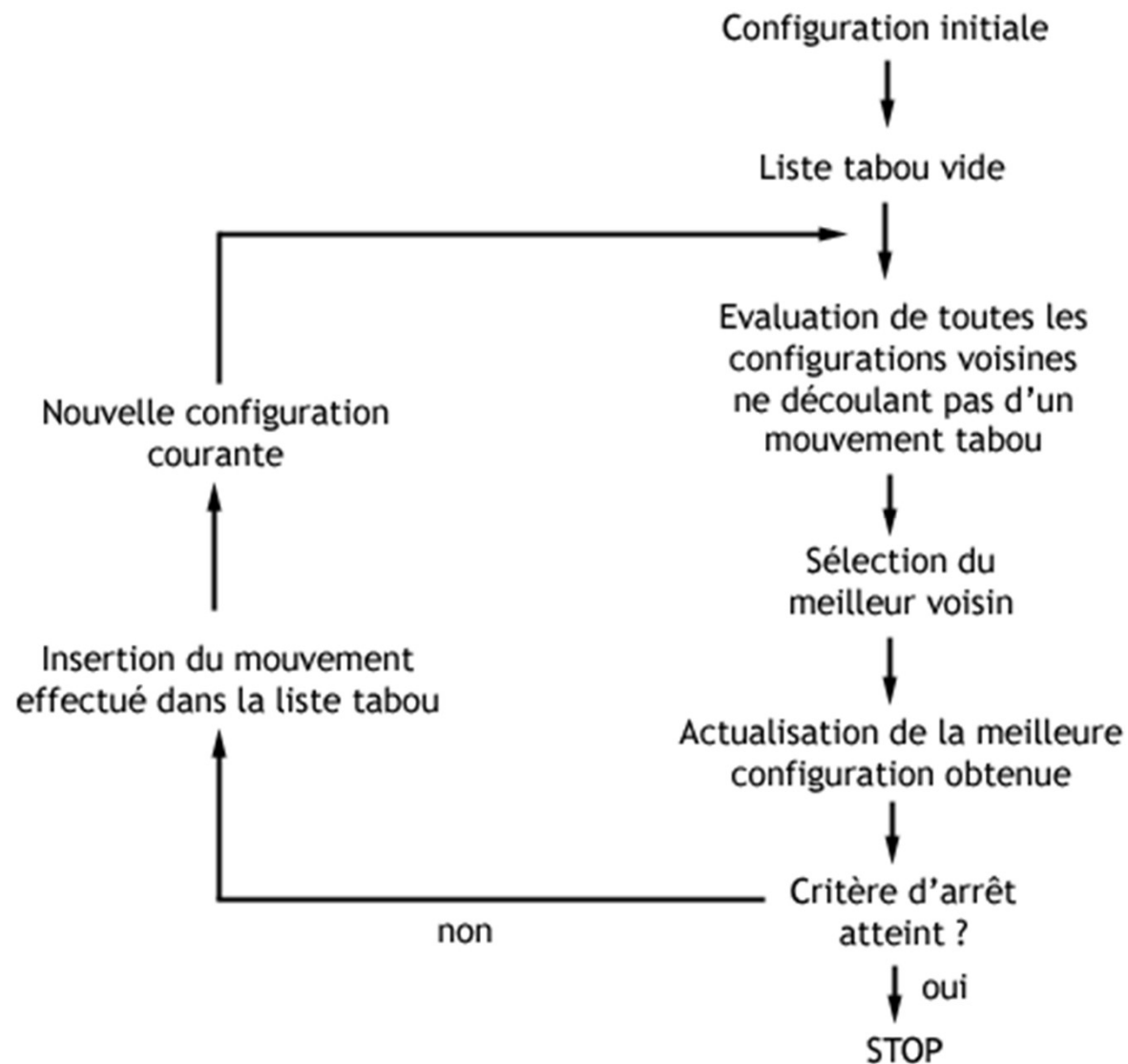
Recherche avec tabou

- Terminologie
 - Mouvement tabou = non souhaitable
 - Par exemple : redescende en un minimum local dont on vient de s'échapper



- Un mouvement est considéré tabou pour un nombre prédéterminé d'itérations

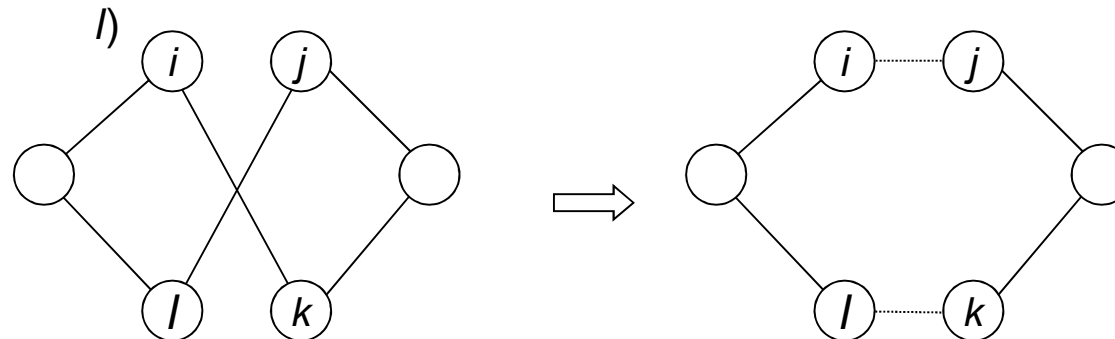
La Recherche Tabou



Heuristiques

Méthodes de recherche locale :

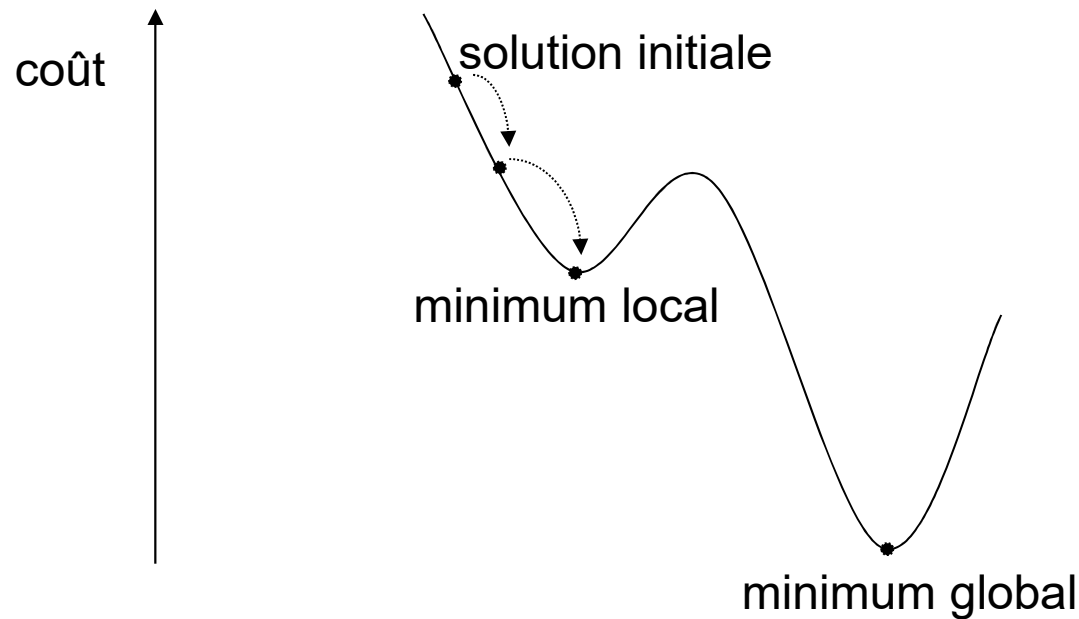
- exemple : méthode d'échange pour le problème du voyageur de commerce (PVC)
 - retirer 2 arêtes (i, k) et (j, l) et les remplacer par les arêtes (i, j) et (k, l)



Heuristiques

Carence des méthodes de recherche locale

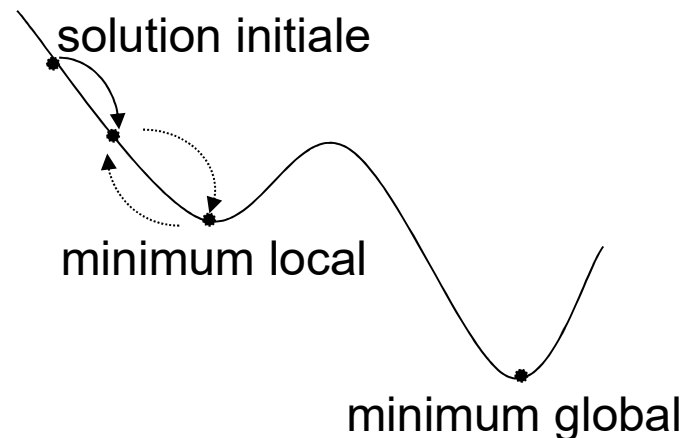
- Sans tabous : incapables de progresser au-delà du premier optimum local rencontré



37

RT : principes de base

- Poursuivre la recherche même s'il y a une dégradation de la valeur de la fonction-objectif
 - risque de cycler

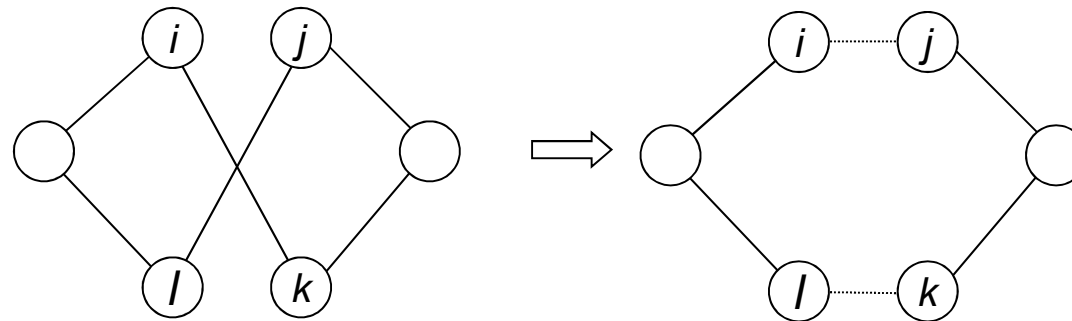


- Solution :
 - mémoriser le cheminement récent de la recherche
 - interdire le retour vers des solutions déjà visitées (tabous)

RT : exemples de tabous

PVC avec voisinage de type « 2-opt » (voir ci-dessous)

- 2-opt constitue l'heuristique interne de l'algorithme de RT
- transformations : retirer les arêtes (i, k) et (j, l) et les remplacer par (i, j) et (k, l)



- tabous possibles :
 - interdire la transformation inverse : $[(i, j), (k, l)] \rightarrow [(i, k), (j, l)]$
 - interdire toute transformation impliquant (i, j) ou (k, l)

RT : mécanismes de tabous

- Mémoire à court terme du processus de recherche
- Possibilités :
 - liste des t dernières transformations effectuées et on interdit la transformation inverse
 - type de tabou le plus fréquent
 - liste des caractéristiques des t dernières solutions ou transformations
 - moins fréquent
 - peut être très efficace

RT : algorithme générique

Soit le problème de minimiser $F(x)$ pour $x \in X$

Posons

- x solution courante
- x^* meilleure solution rencontrée
- T liste des tabous
- $N(x)$ voisinage de x
- $\underline{N}(x)$ voisinage non tabou de x

1. Initialiser

- i. Choisir une solution initiale $x_0 \in X$.
- ii. Poser $x^* = x_0$, $x = x_0$ et $T = \emptyset$.

2. Tant que critère d'arrêt pas satisfait, faire

- i. Déterminer $x' \in \underline{N}(x)$ tel que $F(x') = \min F(x'')$, $x'' \in \underline{N}(x)$. Poser $x = x'$.
- ii. Si $F(x) < F(x^*)$, alors poser $x^* = x$.
- iii. Mettre à jour T .

RT : critère d'aspiration

- But de la mémoire tabou : empêcher de cycler
 - mais les tabous peuvent être trop contraignants :
 - Les solutions attrayantes pourraient être interdites, et ce, même s'il n'y avait pas de risque de cycler
 - Ils pourraient faire stagner la recherche
- Solution : critère d'aspiration (ou fonction d'aspiration)
 - mécanisme inverse qui permet de révoquer le statut TABOU d'une transformation si
 - elle donne une solution intéressante
 - elle n'introduit pas de risque de cycler
- Possibilités :
 - révoquer le tabou si cela améliore la meilleure solution rencontrée
 - règle absolue : si pas de risque de cycler, ignorer le tabou

42

RT : critères d'arrêt

Critères d'arrêt les plus courants :

- dès que la **solution optimale** est obtenue (si connue) ou une solution ayant une valeur prédéterminée
- après un certain **nombre d'itérations** (ou un certain temps de calcul)
- après un certain **nombre d'itérations sans amélioration** de la meilleure solution x^*

RT probabiliste

Dans la version standard de la RT la fonction F doit être évaluée pour chaque voisin de la solution courante x

- peut être très coûteux

Solution considérer un sous-ensemble $N'(x) \subset N(x)$ choisi aléatoirement et choisir la prochaine solution dans $N'(x)$

Avantages

- plus rapide
- échantillon aléatoire : propriété anti-cyclage qui complète la liste des tabous

Désavantages

- on peut manquer de bonnes solutions, voir même l'optimum

RT : gestion dynamique des tabous

Listes traditionnelles

- **circulaires** (FIFO)
- de **longueur fixe**

Désavantages

- liste de longueur fixe n'empêche pas toujours les cycles
- bonne longueur : pas facile à déterminer

Autres méthodes de gestion des tabous

- faire varier la longueur des listes en cours d'exploration
- étiquettes tabou aléatoires
 - durée aléatoire tirée entre $[T_{\min}, T_{\max}]$ à chaque transformation
- liste fixe mais on tire aléatoirement la longueur active de la liste indiquant quels tabous seront maintenus

45

RT : évaluation du voisinage

Dans certains problèmes, la fonction F est très difficile à évaluer (ou très coûteuse)

- évaluation devient prohibitive

Solution

- utiliser une **approximation** de F pour évaluer le voisinage de x
- on ne calculera la vraie valeur de F que pour une transformation choisie ou petit sous-ensemble de candidats prometteurs

Si la plupart des voisins d'une solution ont la même valeur de F , comment en choisir un ?

- utiliser une fonction objectif **auxiliaire** qui mesurera un autre attribut désirable d'une solution

46

RT : conclusion

RT est une approche qui repose sur des concepts simples

Développer un algorithme de RT efficace n'est pas trivial

- quantité de composantes
- flexibilité disponible pour les adapter

Éléments critiques

- calibration des paramètres
- modélisation du problème
 - choix de X
 - choix de $N(x)$ = choix de l'heuristique interne